

SISTEMA DE GESTIÓN HOTELERA "LA GRANGE"

Proyecto Final SQL
Profesora: Nancy Villena
Alumna: Fernanda Barrionuevo
Comisión 96120

ÍNDICE

1. **DEFINICIÓN DEL PROYECTO**
2. **DIAGRAMA DE ENTIDAD RELACIÓN**
3. **LISTADO DE TABLAS**
4. **INCORPORACIÓN DE DATOS**
5. **VISTAS**
6. **FUNCIONES**
7. **STORED PROCEDURE**
8. **TRIGGER**
9. **CONCLUSIÓN**

DEFINIR EL PROYECTO

¿Cuál es el modelo de negocio utilizado?

El modelo es un hotel 3 estrellas de màs de 150 años, se llama “La Grange” la misma se ubica en Courmayeur, norte de Italia, ademàs de brindar su servicio de alojamiento, este cuenta con una granja con una variedad de màs de 100 animales, brindando servicios de paseos, visitas e historia del proyecto, venta de tazones representativos a los diferentes animales, laboratorio y otras actividades como picnics.
También ofrecen servicios de aperitivos (picadas en la tarde que incluyen los animales en la sala de bar) y cena, todo esto con reservaciones previas.

El sistema está orientado a la gestión de un hotel rural que integra:

- Administración de huéspedes
- Gestión de habitaciones y reservas
- Actividades recreativas
- Restaurante y productos
- Control de pagos y facturación

¿Cuál es el objetivo?

El objetivo es introducir organización, planificación y orden a la estructura, brindándole la posibilidad de actualización y hasta un mayor crecimiento en el tiempo. Diseñar e implementar una base de datos eficiente que permita:

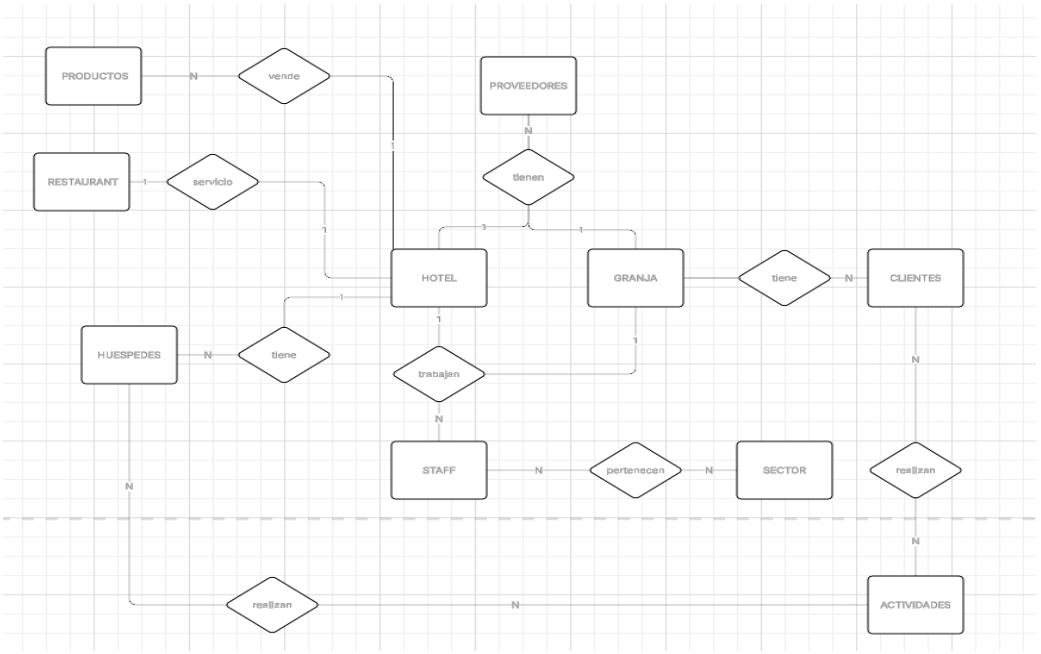
- Organizar la información del hotel
- Automatizar procesos operativos
- Reducir errores en la gestión
- Facilitar el análisis de datos

¿Cuál es la necesidad a cubrir?

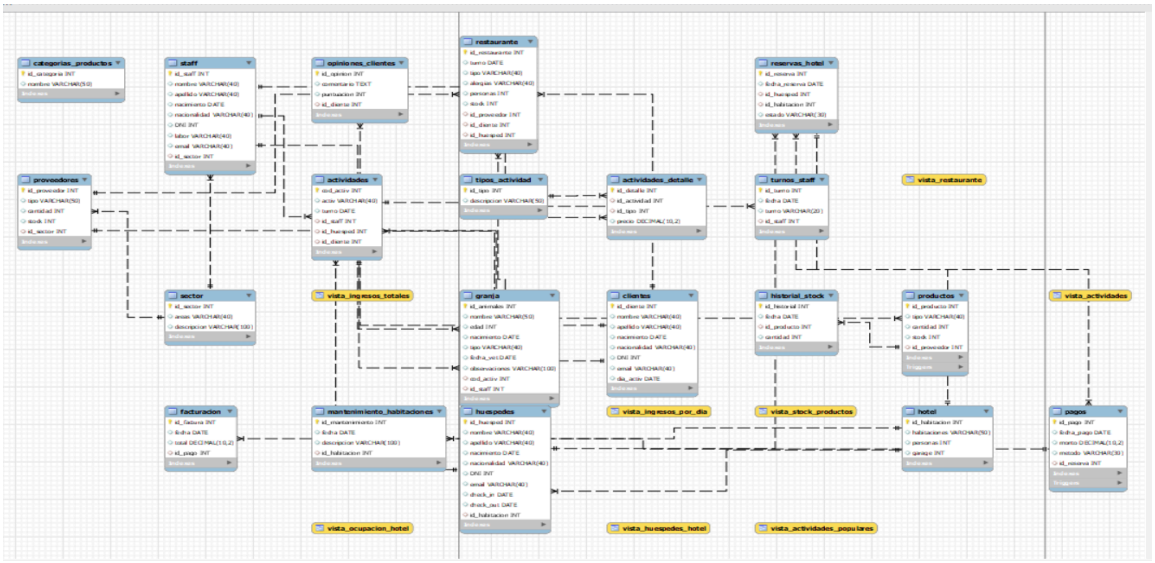
La organaizaciòn inició siendo familiar y pequeña, con el paso de los años ha ido creciendo pero la misma no se ha amoldado tecnològicamente a un programa de organización, más que el básico donde se muestran las planillas de la habitación, esto hace que haya muchos baches, turnos de forma manual y mala administración interna respecto a las visitas o paseos, repercutiendo hasta en errores econòmicos significativos para los dueños.

DIAGRAMA DE ENTIDAD RELACIÓN - INICIACIÓN DEL TRABAJO

El diagrama entidad-relación representa la estructura de la base de datos, mostrando las entidades y sus relaciones. Esta imagen representa la primer parte del trabajo, la misma fue modificada y se amplió la cantidad de tablas utilizadas logrando un análisis más completo y funcional para el hotel.



La segunda captura es la que representa el Diagrama de entidad de relación actual, el mismo contiene el link al sql para poder ver en más detalle la incorporación de las nuevas tablas y el análisis más detallado que se realizó con el avance de la investigación.



SQL - DER

LISTADO DE LAS TABLAS

SECTOR			
Reparto de empleados según sectores			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_sector	-	**int**	PK
areas	-	**varchar(40)**	-
descripcion	-	**varchar(100)**	-

STAFF			
Datos de los empleados			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_staff	-	**int**	PK
nombre	-	**varchar(40)**	-
apellido	-	**varchar(40)**	-
fecha_nacimiento	nacimiento	**date**	-
nacionalidad	-	**varchar(40)**	-
documentacion	DNI	**int**	-
función	labor	**varchar(40)**	-
email	email	**varchar(40)**	-
id_sector		**int**	FK

PROVEEDORES			
Proveedores del hotel y/o granja			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_proveedor	-	**int**	PK
tipo	-	**varchar(50)**	-
cantidad	-	**int**	-
stock	-	**int**	-
id_sector		**int**	FK

HOTEL			
Datos principales del hotel			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_habitacion	-	**int**	PK
habitaciones		**varchar(50)**	
cantidad de personas	personas	**int**	-
estacionamiento	garage	**int**	-

CLIENTES			
Datos de los clientes			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_cliente	-	**int**	PK
nombre	-	**varchar(40)**	-
apellido	-	**varchar(40)**	-
fecha_nacimiento	nacimiento	**date**	-
nacionalidad		**varchar(40)**	-
documentacion	DNI	**int**	-
email	email	**varchar(40)**	-
día de actividad	dia_activ	**date**	-

HUESPEDES			
Datos de los huespedes			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_huesped	-	**int**	PK
nombre	-	**varchar(40)**	-
apellido	-	**varchar(40)**	-
fecha_nacimiento	nacimiento	**date**	-
nacionalidad		**varchar(40)**	-
documentacion	DNI	**int**	-
email	email	**varchar(40)**	-
entrada	check in	**date**	-
salida	check out	**date**	-
id_habitacion	-	**int**	FK

ACTIVIDADES			
Datos correspondientes a las actividades granja			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
codigo actividad	cod_activ	**int**	PK
actividad	activ	**varchar(40)**	-
turnos	turno	**date**	-
id_staff	-	**int**	FK
id_huesped	-	**int**	FK
id_cliente	-	**int**	FK

GRANJA			
Datos correspondientes a la granja			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_animales	-	**int**	PK
nombre	-	**varchar(40)**	-
edad	-	**int**	-
fecha_nacimiento	nacimiento	**date**	
tipo	tipo	**varchar(40)**	
revisión veterinaria	fecha_vet	**date**	
obsevaciones	-	**varchar(100)**	
codigo actividad	cod_activ	**int**	FK
id_staff		**int**	FK

PRODUCTOS			
Ventas de productos			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_producto	-	**int**	PK
tipo	-	**varchar(40)**	-
cantidad	-	**int**	-
stock	-	**int**	-
id_proveedor		**int**	FK

RESTAURANTE			
Reservas cena			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_restaurante	-	**int**	PK
turno	-	**date**	-
tipo	-	**varchar(40)**	-
aclaración	alergias	**varchar(40)**	-
cantidad de personas	personas	**int**	-
stock		**int**	
id_proveedor		**int**	FK
id_cliente		**int**	FK
id_huesped		**int**	FK

RESERVA HOTEL			
Reservas hotel por plataformas			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_reserva	-	**int**	PK
fecha_reserva	-	**date**	-
estado	-	**varchar(30)**	-
id_huesped	-	**int**	FK
id_habitacion	-	**int**	FK

PAGOS			
Pagos de las reservas			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_pago	-	**int**	PK
fecha_pago	-	**date**	-
monto	-	**decimal(10,2)**	-
metodo	-	**varchar(30)**	-
id_reserva	-	**int**	FK

FACTURACION			
Total facturación diaria			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_factura	-	**int**	PK
fecha	-	**date**	-
total	-	**decimal(10,2)**	-
id_pago	-	**int**	FK

TIPOS ACTIVIDAD			
Actividades que se prestan en el hotel			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_detalle	-	**int**	PK
id_actividad	-	**int**	FK
id_tipo	-	**int**	FK
precio	-	**decimal(10,2)**	-

CATEGORIAS PRODUCTOS			
Diversificacion de productos			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_categoria	-	**int**	PK
nombre	-	**varchar(50)**	

HISTORIAL STOCK			
Información comparativa de historial stock por temporada laboral			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_historial	-	**int**	PK
fecha	-	**date**	-
id_producto	-	**int**	FK
cantidad	-	**int**	-

TURNO STAFF			
Turnos staff del hotel			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_turno	-	**int**	PK
fecha	-	**date**	-
turno	-	**varchar(20)**	-
id_staff	-	**int**	FK

MANTENIMIENTO HABITACIONES			
Mantenimiento y acondicionamiento de las habitaciones			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_mantenimiento	-	**int**	PK
fecha	-	**date**	-
descripcion	-	**varchar(100)**	-
id_habitaciones	-	**int**	FK

OPINIONES CLIENTES			
Reseñas de los clientes			
nombre de campo	**abreviatura**	**tipo de datos**	**tipo de claves**
id_opiniones	-	**int**	PK
comentarios	-	**text**	-
puntuación	-	**int**	-
id_cliente	-	**int**	FK

INSERCIÓN DE DATOS

Se realizó la carga de datos en las distintas tablas de la base de datos mediante la sentencia INSERT INTO.

```sql
INSERT INTO sector (areas, descripcion) VALUES
('Recepción', 'Atención al cliente'),
('Granja', 'Cuidado de animales'),
('Restaurante', 'Servicio de comidas');


INSERT INTO staff (nombre, apellido, nacimiento, nacionalidad, DNI, labor, email, id_sector) VALUES
('Ana', 'Lopez', '1990-05-10', 'Española', 12345678, 'Recepcionista', 'ana@gmail.com', 1),
('Juan', 'Perez', '1985-08-20', 'Argentina', 23456789, 'Granjero', 'juan@mail.com', 2);


INSERT INTO hotel (habitaciones, personas, garage) VALUES
('Suite 1', 2, 1),
('Habitación Familiar', 4, 1);


INSERT INTO clientes (nombre, apellido, nacimiento, nacionalidad, DNI, email, dia_activ) VALUES
('Laura', 'Gomez', '1995-06-15', 'Española', 34567890, 'laura@gmail.com', '2026-03-20');
```

Datos creados de forma manual con el objetivo de simular el funcionamiento real del sistema del hotel, incluyendo información de sectores, empleados, huéspedes, clientes, actividades, productos y reservas al restaurante por ej.
El objetivo de esto es:

- Permitir la ejecución de consultas reales
- Probar el funcionamiento de las vistas, funciones, stored procedure y triggers.
- Simular un entorno operativo del hotel con información coherente

```sql
INSERT INTO clientes (nombre, apellido, nacimiento, nacionalidad, DNI, email, dia_activ) VALUES
('Laura', 'Gomez', '1995-06-15', 'Española', 34567890, 'laura@gmail.com', '2026-03-20');


INSERT INTO huespedes (nombre, apellido, nacimiento, nacionalidad, DNI, email, check_in, check_out, id_habitacion) VALUES
('Carlos', 'Ruiz', '1988-02-10', 'Española', 45678901, 'carlos@gmail.com', '2026-03-01', '2026-03-05', 1);


INSERT INTO actividades (activ, turno, id_staff, id_huesped, id_cliente) VALUES
('Visita a la granja', '2026-03-02', 2, 1, 1);


INSERT INTO granja (nombre, edad, nacimiento, tipo, fecha_vet, observaciones, cod_activ, id_staff) VALUES
('Vaca Lola', 5, '2020-01-01', 'Vaca', '2026-01-10', 'Saludable', 1, 2);


INSERT INTO proveedores (tipo, cantidad, stock, id_sector) VALUES
('Alimentos', 100, 50, 3);
```

Se insertaron datos en las siguientes tablas:

- sector
- staff
- hotel
- clientes
- huéspedes
- actividades
- granja
- proveedores

- productos
- restaurante

Los datos que se incluyen:

- Información personal (nombre, apellido, DNI, email)
- Datos de alojamiento (habitaciones, fechas de check-in y check-out)
- Actividades de la granja
- Información del personal del hotel
- Productos y proveedores
- Reservas del restaurante

Se respetaron las relaciones entre tablas mediante el uso de claves foráneas. Los datos fueron insertados en un orden específico para evitar errores de integridad referencial. La carga de datos permitió validar el correcto funcionamiento de las relaciones entre entidades.

```sql
INSERT INTO restaurante (turno, tipo, alergias, personas, stock, id_proveedor, id_cliente, id_huesped) VALUES
('2026-03-03', 'Cena', 'Ninguna', 2, 5, 1, 1, 1);

INSERT INTO reservas_hotel (fecha_reserva, id_huesped, id_habitacion, estado) VALUES
('2026-03-01', 1, 1, 'Confirmada'),
('2026-03-02', 1, 2, 'Pendiente');

INSERT INTO pagos (fecha_pago, monto, metodo, id_reserva) VALUES
('2026-03-01', 200.50, 'Tarjeta', 1),
('2026-03-02', 150.00, 'Efectivo', 2);

INSERT INTO facturacion (fecha, total, id_pago) VALUES
('2026-03-01', 200.50, 1),
('2026-03-02', 150.00, 2);
```

```sql
INSERT INTO actividades_detalle (id_actividad, id_tipo, precio) VALUES
(1, 1, 50.00),
(1, 2, 30.00);

INSERT INTO categorias_productos (nombre) VALUES
('Lácteos'),
('Souvenirs');

INSERT INTO historial_stock (fecha, id_producto, cantidad) VALUES
('2026-03-01', 1, 10),
('2026-03-02', 1, -5);

INSERT INTO turnos_staff (fecha, turno, id_staff) VALUES
('2026-03-01', 'Mañana', 1),
('2026-03-01', 'Tarde', 2);
```

```sql
INSERT INTO mantenimiento_habitaciones (fecha, descripcion, id_habitacion) VALUES
('2026-03-01', 'Revisión general', 1);

INSERT INTO opiniones_clientes (comentario, puntuacion, id_cliente) VALUES
('Excelente servicio', 5, 1),
('Muy buena experiencia', 4, 1);
```

VISTAS

1) **Huéspedes del Hotel**

Descripción: Esta vista muestra la información de los huéspedes junto con la habitación en la que se alojan y las fechas de su estadía (check-in y check-out).

Objetivo: Facilitar la consulta de ocupación del hotel, permitiendo visualizar rápidamente qué huéspedes están alojados y en qué habitaciones.

Tablas utilizadas:

- huéspedes
- hotel

```
CREATE VIEW vista_huespedes_hotel AS
SELECT
    h.nombre,
    h.apellido,
    h.check_in,
    h.check_out,
    ho.habitaciones
FROM huespedes h
JOIN hotel ho ON h.id_habitacion = ho.id_habitacion;
```

2) **Actividades**

Descripción: Esta vista reúne la información de las actividades realizadas en la granja, incluyendo el personal encargado, los clientes y los huéspedes que participan.

Objetivo: Permitir una visualización completa de las actividades del hotel, facilitando la gestión y organización de las mismas.

Tablas utilizadas:

- actividades
- staff
- clientes
- huéspedes

```sql
CREATE VIEW vista_actividades AS
SELECT
    a.activ,
    a.turno,
    s.nombre AS staff_nombre,
    c.nombre AS cliente_nombre,
    h.nombre AS huesped_nombre
FROM actividades a
LEFT JOIN staff s ON a.id_staff = s.id_staff
LEFT JOIN clientes c ON a.id_cliente = c.id_cliente
LEFT JOIN huespedes h ON a.id_huesped = h.id_huesped;
```

3) Restaurante

Descripción: Esta vista muestra las reservas del restaurante junto con la información de los clientes y huéspedes asociados.

Objetivo: Facilitar el control de reservas y la planificación del servicio gastronómico.

Tablas utilizadas:

- restaurante
- clientes
- huéspedes

```sql
CREATE VIEW vista_restaurante AS
SELECT
    r.tipo,
    r.turno,
    r.personas,
    c.nombre AS cliente,
    h.nombre AS huesped
FROM restaurante r
LEFT JOIN clientes c ON r.id_cliente = c.id_cliente
LEFT JOIN huespedes h ON r.id_huesped = h.id_huesped;
```

4) Ingresos totales

Descripción: Esta vista calcula el total de ingresos generados por el hotel a partir de los pagos registrados en el sistema.

Objetivo: Permitir obtener una visión general del rendimiento económico del hotel en un solo valor consolidado.

Tablas utilizadas:

- pagos

```
312 •   CREATE VIEW vista_ingresos_totales AS
313     SELECT
314         SUM(monto) AS total_ingresos
315     FROM pagos;
```

5) Ingresos por día

Descripción: Esta vista muestra los ingresos agrupados por fecha, permitiendo visualizar cuánto dinero se genera cada día.

Objetivo: Facilitar el análisis temporal de los ingresos y detectar tendencias o variaciones en el flujo económico.

Tablas utilizadas:

- pagos

```
316 •   CREATE VIEW vista_ingresos_por_dia AS
317     SELECT
318         fecha_pago,
319         SUM(monto) AS total_dia
320     FROM pagos
321     GROUP BY fecha_pago;
```

6) Actividades populares

Descripción: Esta vista muestra las actividades disponibles junto con la cantidad de veces que han sido realizadas.

Objetivo: Identificar cuáles son las actividades más demandadas por los clientes y huéspedes.

Tablas utilizadas:

- actividades

```sql
CREATE VIEW vista_actividades_populares AS
SELECT
    activ,
    COUNT(*) AS cantidad
FROM actividades
GROUP BY activ;
```

7) Ocupación hotel

Descripción: Esta vista presenta la cantidad de huéspedes asignados a cada habitación del hotel.

Objetivo: Permitir el control de la ocupación del hotel y facilitar la gestión de disponibilidad de habitaciones.

Tablas utilizadas:

- hotel
- huéspedes

```sql
CREATE VIEW vista_ocupacion_hotel AS
SELECT
    ho.habitaciones,
    COUNT(h.id_huesped) AS ocupacion
FROM hotel ho
LEFT JOIN huespedes h ON ho.id_habitacion = h.id_habitacion
GROUP BY ho.habitaciones;
```

8) Stock productos

Descripción: Esta vista muestra el stock total de productos calculado a partir del historial de movimientos registrados.

Objetivo: Facilitar el control del inventario y asegurar la disponibilidad de productos.

Tablas utilizadas:

- productos
- historial_stock

```sql
CREATE VIEW vista_stock_productos AS
SELECT
    p.tipo,
    SUM(hs.cantidad) AS stock_total
FROM productos p
JOIN historial_stock hs ON p.id_producto = hs.id_producto
GROUP BY p.tipo;
```

FUNCIONES

1) TOTAL HUÉSPEDES

Descripción:
Esta función devuelve el número total de huéspedes registrados en la base de datos.

Objetivo:
Permitir obtener rápidamente una métrica general de ocupación del hotel.

Tablas utilizadas:

- huéspedes

```sql
DELIMITER //
CREATE FUNCTION total_huespedes()
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*) INTO total
    FROM huespedes;

    RETURN total;
END //

DELIMITER ;
```

2) ACTIVIDADES POR CLIENTE

Descripción:
Esta función recibe como parámetro el ID de un cliente y devuelve la cantidad de actividades en las que ha participado.

Objetivo:
Facilitar el análisis de participación de los clientes en las actividades ofrecidas por el hotel.

Tablas utilizadas:

- actividades

```
CREATE FUNCTION actividades_por_cliente(id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*) INTO total
    FROM actividades
    WHERE id_cliente = id;

    RETURN total;
END //
DELIMITER ;
```

3) CALCULAR TOTAL PAGOS

Descripción:
Esta función permite calcular el total de dinero generado a partir de los pagos registrados en el sistema.

Objetivo:
Facilitar la obtención de un valor consolidado de ingresos, permitiendo reutilizar esta lógica en consultas y reportes.

Tablas utilizadas:

- pagos

```sql
DELIMITER //

CREATE FUNCTION calcular_total_pagos()
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE total DECIMAL(10,2);

    SELECT SUM(monto) INTO total
    FROM pagos;

    RETURN total;
END //

DELIMITER ;
```

4) CONTAR RESERVAS CLIENTE

Descripción: Esta función devuelve la cantidad de reservas realizadas por un cliente específico.

Objetivo: Permitir el análisis del comportamiento de los clientes, identificando frecuencia de uso del servicio.

Tablas utilizadas:

- reservas_hotel
- clientes

```sql
DELIMITER //

CREATE FUNCTION contar_reservas_cliente(p_id_cliente INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*) INTO total
    FROM reservas_hotel
    WHERE id_cliente = p_id_cliente;

    RETURN total;
END //

DELIMITER ;
```

STORED PROCEDURE

1) Agregar_cliente

Descripción:
Este procedimiento permite insertar un nuevo cliente en la base de datos mediante el ingreso de sus datos personales.

Objetivo:
Automatizar el registro de clientes, evitando errores manuales y simplificando el proceso de carga de datos.

Tablas utilizadas:

- clientes

```
424     DELIMITER //
425
426 •   CREATE PROCEDURE agregar_cliente (
427         IN p_nombre VARCHAR(40),
428         IN p_apellido VARCHAR(40),
429         IN p_nacimiento DATE,
430         IN p_nacionalidad VARCHAR(40),
431         IN p_DNI INT,
432         IN p_email VARCHAR(40),
433         IN p_dia_activ DATE
434     )
435     BEGIN
436         INSERT INTO clientes (nombre, apellido, nacimiento, nacionalidad, DNI, email, dia_activ)
437         VALUES (p_nombre, p_apellido, p_nacimiento, p_nacionalidad, p_DNI, p_email, p_dia_activ);
438     END //
439
440     DELIMITER ;
```

2) Ver_actividades_clientes

Descripción:
Este procedimiento recibe como parámetro el ID de un cliente y devuelve todas las actividades en las que ha participado.

Objetivo:
Permitir consultas específicas sobre la actividad de un cliente dentro del sistema.

Tablas utilizadas:

- actividades

```
DELIMITER //

CREATE PROCEDURE ver_actividades_cliente (
    IN p_id_cliente INT
)
BEGIN
    SELECT *
    FROM actividades
    WHERE id_cliente = p_id_cliente;
END //

DELIMITER ;
```

3) Registrar_pago

Descripción: Este procedimiento permite registrar un nuevo pago en la base de datos.

Objetivo: Automatizar la inserción de pagos y garantizar la consistencia en los datos registrados.

Tablas utilizadas:

- pagos

```
459     DELIMITER //
460
461 •   CREATE PROCEDURE registrar_pago(
462         IN p_fecha DATE,
463         IN p_monto DECIMAL(10,2),
464         IN p_metodo VARCHAR(50),
465         IN p_id_reserva INT
466     )
467     BEGIN
468         INSERT INTO pagos (fecha_pago, monto, metodo, id_reserva)
469         VALUES (p_fecha, p_monto, p_metodo, p_id_reserva);
470     END //
471
472     DELIMITER ;
473
474 •   CALL registrar_pago('2026-03-10', 500, 'Tarjeta', 1);
```

4) **Reservas_por_cliente**

Descripción: Este procedimiento permite obtener todas las reservas realizadas por un cliente específico.

Objetivo: Facilitar la consulta de información relacionada con clientes y su historial de reservas.

Tablas utilizadas:

- reservas_hotel

```
478     DELIMITER //
479
480 ●   CREATE PROCEDURE reservas_por_cliente(
481         IN p_id_cliente INT
482     )
483     BEGIN
484         SELECT r.*
485         FROM reservas_hotel r
486         JOIN huespedes h ON r.id_huesped = h.id_huesped
487         WHERE h.id_cliente = p_id_cliente;
488     END //
489
490     DELIMITER ;
```

TRIGGER

1) **Actualizar_stock_proveedor**

Descripción:
Este trigger se ejecuta automáticamente después de insertar un nuevo producto en la tabla productos. Su función es actualizar el stock del proveedor correspondiente.

Objetivo:
Mantener actualizado el inventario de los proveedores de manera automática, evitando inconsistencias en los datos.

Tablas utilizadas:

- productos
- proveedores

```
CREATE TRIGGER actualizar_stock_proveedor
AFTER INSERT ON productos
FOR EACH ROW
BEGIN
    UPDATE proveedores
    SET stock = stock + NEW.cantidad
    WHERE id_proveedor = NEW.id_proveedor;
END //

DELIMITER ;
```

```
INSERT INTO productos (tipo, cantidad, stock, id_proveedor)
VALUES ('Leche', 10, 0, 1);
```

```
SELECT * FROM proveedores;
```

2) **Generar_factura**

Descripción:
Este trigger se ejecuta automáticamente después de registrar un nuevo pago. Su función es generar una factura asociada a dicho pago.

Objetivo:
Automatizar el proceso de facturación, evitando registros manuales y asegurando la consistencia entre pagos y facturas.

Tablas utilizadas:

- pagos
- facturación

```sql
CREATE TRIGGER generar_factura
AFTER INSERT ON pagos
FOR EACH ROW
BEGIN
    INSERT INTO facturacion (fecha, total, id_pago)
    VALUES (NEW.fecha_pago, NEW.monto, NEW.id_pago);
END //

DELIMITER ;

INSERT INTO pagos (fecha_pago, monto, metodo, id_reserva)
VALUES ('2026-03-05', 300.00, 'Tarjeta', 1);

SELECT * FROM facturacion;
```

CONCLUSIÓN

El desarrollo del presente proyecto permitió la creación de una base de datos relacional completa, orientada a la gestión integral de un hotel rural de Italia. A través de su implementación, se logró centralizar la información de huéspedes, reservas, actividades, pagos y productos, facilitando la organización y el acceso a los datos.

El sistema está diseñado para ser utilizado por distintos perfiles dentro de la organización, como el personal administrativo, el área de recepción, el sector contable y los responsables de gestión. Cada uno de estos usuarios puede acceder a información relevante para sus tareas, mejorando la eficiencia operativa y reduciendo errores en los procesos diarios.

Asimismo, se incorporaron mecanismos de control como el uso de Transaction Control Language (TCL), permitiendo asegurar la integridad de los datos mediante el manejo de transacciones, y Data Control Language (DCL), orientado a la gestión de permisos y accesos dentro de la base de datos, fortaleciendo la seguridad del sistema.

Por otro lado, la implementación de vistas, funciones, procedimientos almacenados y triggers permitió automatizar procesos clave y simplificar consultas complejas, aportando robustez y escalabilidad a la solución desarrollada.

Finalmente, se realizó un análisis de los datos utilizando Microsoft Excel, a partir del cual se generaron visualizaciones que permiten interpretar la información almacenada, identificar patrones de ingresos y comportamiento de los clientes, y facilitar la toma de decisiones estratégicas.

En conclusión, este proyecto no solo permite gestionar eficientemente la información del hotel, sino también transformarla en conocimiento útil, constituyendo una herramienta fundamental para la optimización del negocio y su crecimiento futuro.